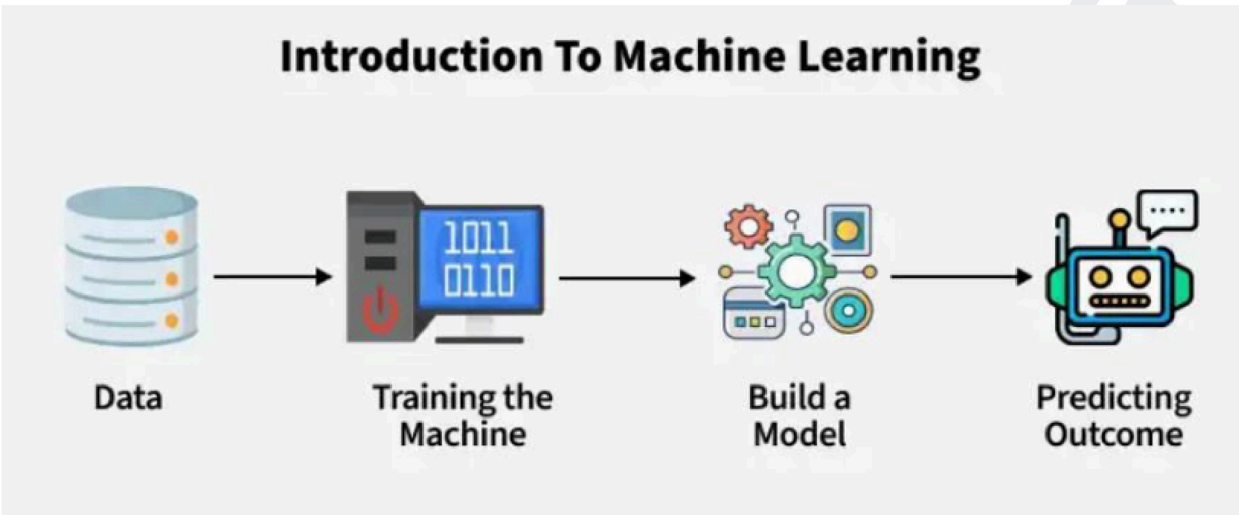
MACHINE LEARNING

Machine Learning (ML) is a branch of artificial intelligence (AI) that enables computers to learn from data and make decisions or predictions without being explicitly programmed. Instead of following a fixed set of instructions, ML models identify patterns in data and improve their performance over time



- Automation: Reduces human effort by automating tasks
- Data-Driven Decisions: Helps in making accurate predictions
- Improves Over Time: The model learns and enhances its accuracy with more data.

Machine Learning (ML) Pipeline

An ML pipeline is a structured sequence of steps **used to develop, train, and deploy machine learning models efficiently**. It ensures a smooth workflow, from raw data processing to model deployment.

Key Stages of an ML Pipeline

1. Data Collection

- Sources: Databases

• Formats: CSV, JSON, Images, Videos
• Tools: Pandas

2. Data Preprocessing

• Handling Missing Values **(Imputation, Deletion)**
• Feature Scaling **(Standardization, Normalization)**
• Encoding Categorical Data **(One-Hot Encoding, Label Encoding)**
• Outlier Detection & Treatment **(Z-score, IQR method)**

3. Feature Engineering

• Feature Selection **(Removing irrelevant features)**
• Feature Extraction
• Feature Transformation

4. Splitting Data

• Train-Test Split (80-20, 70-30, etc.)
• Cross-Validation (K-Fold, Stratified K-Fold, Leave-One-Out CV)

5. Model Selection

• Algorithms : Simple Linear, Polynomial, Knn, Decision Trees, SVR, Random Forest, XGBoost, Multiple Linear
• Hyperparameter Tuning : GridSearchCV, RandomizedSearchCV

6. Model Evaluation

Classification Metrics : Accuracy, Precision, Recall, F1-Score, ROC-AUC
Regression Metrics : MSE, RMSE, R^2 Score

7. Model Deployment

Flask / FastAPI for Web Deployment
Streamlit for Interactive Apps
Docker & Kubernetes for Scaling

3. ML Pipeline Flowchart

Data Collection → Preprocessing → Feature Engineering → Splitting Data → Model Training → Evaluation → Deployment

Regression Problem in General

Regression is a type of **supervised learning** problem where the goal is **to predict a continuous numerical value based on input features**. The output is a real number, and the **model learns the relationship between the input variables and the target variable.**

Key Characteristics:

- Output : Continuous value (e.g., price, temperature, age).
- Objective : Minimize the error between predicted and actual values.
- Evaluation Metrics : Mean Squared Error (MSE), Root Mean Squared Error (RMSE), Mean Absolute Error (MAE), R-squared.

Examples:
- Predicting house prices based on features like size, location, and number of rooms.
- Forecasting stock prices or sales.
- Estimating the temperature based on weather data.

Common Algorithms:

- Linear Regression
- Polynomial Regression

- Ridge Regression
- Lasso Regression
- Support Vector Regression (SVR)
- Knn
- Decision Tree Regression
- Random Forest

Regression Algorithms

1. Simple Linear Regression

Models the relationship between a single independent variable XX and a dependent variable Y using a straight line.

$$Y = \beta_0 + \beta_1 X + \epsilon$$

2. Multiple Linear Regression

Extends simple linear regression to model the relationship between multiple independent variables X1,X2,…,Xn and a dependent variable Y

$$Y = \beta_0 + \beta_1 X_1 + \beta_2 X_2 + \cdots + \beta_n X_n + \epsilon$$

3. Polynomial Regression

Models the relationship between X and Y as an n-degree polynomial. It is useful when the relationship is non-linear.

$$Y = \beta_0 + \beta_1 X + \beta_2 X^2 + \cdots + \beta_n X^n + \epsilon$$

4. Lasso Regression (L1 Regularization)

Adds an L1 penalty term to the loss function to shrink less important feature coefficients to zero, effectively performing feature selection

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^{n} |\beta_i|$$

5. Ridge Regression (L2 Regularization)

Adds an L2 penalty term the loss function to shrink the coefficients, reducing model complexity and preventing overfitting

$$\text{Loss} = \text{MSE} + \lambda \sum_{i=1}^{n} \beta_i^2$$

6. Elastic Net Regression

Combines L1 and L2 regularization to balance feature selection and coefficient shrinkage.

$$\text{Loss} = \text{MSE} + \lambda_1 \sum_{i=1}^{n} |\beta_i| + \lambda_2 \sum_{i=1}^{n} \beta_i^2$$

Steps to Perform Regression Analysis

1. Data Collection:

Gather the dataset containing the dependent and independent variables.

2. Data Preprocessing:

Handle missing values, outliers, and categorical variables.
Normalize or standardize the features if necessary.

3. Model Selection:

Choose the appropriate regression algorithm based on the problem and data.

4. Model Training:

Split the data into training and testing sets.
Train the model on the training set.

5. Model Evaluation:

Evaluate the model using metrics like:

- Mean Squared Error (MSE)
- R-squared (Coefficient of Determination)
- Mean Absolute Error (MAE)

6. Model Tuning:

Adjust hyperparameters (e.g., regularization strength) to improve performance.

7. Prediction:

Use the trained model to make predictions on new data.

Quick Reference: ML Algorithms Cheat Sheet

Problem Type	Recommended Algorithm
Linear Relationship	Linear / Logistic Regression
Non-linear, Interpretable	Decision Trees
High Accuracy Needed	Random Forest / XGBoost
Text Classification	Naive Bayes, Transformers
Image Recognition	CNN, Transfer Learning
Sequence Data	RNN, LSTM, Transformers
Small Dataset (<1000)	Simple models, SVM
Large Dataset (>1M)	Neural Networks, XGBoost
Real-time Predictions	Optimized trees, LightGBM
Feature Selection Needed	Lasso, Random Forest